

Name - Chaitanya Kelkar

Roll no - 15

Experiment No 1

Aim - Cryptography in Blockchain, Merkle root Tree Hash

Theory -

1. Cryptographic Hash Functions

A Cryptographic Hash Function (e.g., SHA-256) converts an input of any size into a fixed-length hexadecimal string.

- One-Way: Computationally impossible to reverse-engineer the original input from the hash.
- Deterministic: Identical inputs always produce the identical output.
- Avalanche Effect: Changing a single character completely alters the resulting hash.

2. Merkle Tree Architecture

A Merkle Tree is a binary hash tree used to structure and summarize block data efficiently:

- Leaf Nodes: Store individual hashes of transaction data.
- Non-Leaf Nodes: Contain the combined hash of their two child nodes ($\text{Hash}(A + B)$).
- Merkle Root: The single hash at the top representing the cryptographic signature of all transactions in the block.

3. Cryptographic Puzzle & Nonce

- Nonce: A *"Number used ONCE"* modified by miners to change a block's hash.
- Puzzle: Finding a nonce such that $\text{SHA-256}(\text{Block Data} + \text{Nonce}) \leq \text{Target}$.
- Golden Nonce: The specific nonce value that successfully satisfies the network's difficulty target.

Input and Output -

Program 1: Python program that uses the hashlib library to create the hash of a given string

Input -

```

import hashlib          #corrected statement - provides a unified
                        interface for generating cryptographic hashes and message digests

def create_hash(string):

    # Create a hash object using SHA-256 algorithm

    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object

    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash

    hash_string = hash_object.hexdigest()
#corrected statement - referenced hash_object

    # Return the hash string

    return hash_string


# Example usage

input_string = input("Enter a string: ")

hash_result = create_hash(input_string)

print("Hash:", hash_result)
#corrected statement - printed hash_result

```

Output -

Enter a string: chaitanya

Hash: c70cd83c6c76a7ff3a62ab8f2655e534bc6bdf55b36cac02b5958f7a596ed5bb

Program 2: Program to generate required target hash with input string and nonce

Input -

```

import hashlib

# Get user input

input_string = input("Enter a string: ")

nonce = input("Enter the nonce: ")

```

```

# Concatenate the string and nonce
hash_string = input_string + nonce          # corrected - added nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)             # corrected - printed hash code

```

Output -

```

Enter a string: chaitanya
Enter the nonce: 15

Hash Code:
dfc3eacac83179a0cfc00c4a577a82a68ae8d11e1b2f165dce313d28ec01f5be

```

Program 3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

Input -

```

import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256

```

```

hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Check if the hash code has the required number of leading
zeros

if hash_code.startswith(hash_prefix):
    print("Hash:", hash_code )
    return nonce

nonce += 1

# Get user input
input_string = "chaitanya"
num_zeros = 4

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

Output -

```

Hash: 00002cfb31c95723c237eec98cbc5b04065b3df20a5a2f3bd8faa5aa4134b3db

Input String: chaitanya

Leading Zeros: 4

Expected Nonce: 131712

```

Program 4: Generating Merkle Tree for given set of Transactions

Input -

```
import hashlib

def hash_data(data):
    """Utility function to calculate SHA-256 hash in hex format."""
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def build_merkle_tree(transactions):
    if not transactions:
        return None

    # Step 1: Hash raw transactions to generate initial leaf nodes
    print("=== Level 0: Leaf Hashes (Hashed Transactions) ===")
    current_level = []

    for tx in transactions:
        tx_hash = hash_data(tx)
        current_level.append(tx_hash)
        print(f"Transaction: '{tx}'")
        print(f"    -> Hash: {tx_hash}\n")

    level_num = 1

    # Step 2: Combine hashes level by level until root is reached
    while len(current_level) > 1:
        # If odd number of hashes, duplicate the last element to form a
        pair

        if len(current_level) % 2 != 0:
            print(f"[!] Odd count at Level {level_num - 1}. Duplicating
last hash: {current_level[-1]}\n")
            current_level.append(current_level[-1])
```

```

print(f"=== Level {level_num}: Intermediate Hashes ===")
next_level = []

for i in range(0, len(current_level), 2):
    left = current_level[i]
    right = current_level[i + 1]
    combined = left + right
    combined_hash = hash_data(combined)

    next_level.append(combined_hash)

    print(f"Pairing:")
    print(f"  Left : {left}")
    print(f"  Right: {right}")
    print(f"  Combined Hash -> {combined_hash}\n")

current_level = next_level
level_num += 1

# Final Merkle Root
merkle_root = current_level[0]
return merkle_root

# --- Input Data ---
transactions = [
    "Alice -> Bob : $300",
    "Bob -> Dave : $600",
    "Dave -> Eve : $200",

```

```

    "Eve -> Alice : $400",
    "Roo -> Bob : $10"
]

# Run the Merkle Tree construction
merkle_root = build_merkle_tree(transactions)

print("=====")
print("FINAL MERKLE ROOT HASH:", merkle_root)
print("=====")

```

Output -

```

=== Level 0: Leaf Hashes (Hashed Transactions) ===

Transaction: 'Alice -> Bob : $300'

-> Hash:
2abdb440fb993eeb60cc103a6cf13548155a228aedd52a35a9b063a641a85d4f

Transaction: 'Bob -> Dave : $600'

-> Hash:
568f0248df3072297a3378af99c745fbca01267784a32b02eced2c2d8d0aaee1

Transaction: 'Dave -> Eve : $200'

-> Hash:
277bc788db649e93f8758a0fae636b92144adb5150a5fb82d98ac2c95ceae07f

Transaction: 'Eve -> Alice : $400'

-> Hash:
fa69ffb869386437ea2645096d3e872652a01f9b65cfb52ff548a8ddf382ec50

Transaction: 'Roo -> Bob : $10'

```

-> Hash:
9e9a4c4243de110eb4b232d912351725ee6fcf1028f4f75638438ee83061feca

[!] Odd count at Level 0. Duplicating last hash:
9e9a4c4243de110eb4b232d912351725ee6fcf1028f4f75638438ee83061feca

=== Level 1: Intermediate Hashes ===

Pairing:

Left :
2abdb440fb993eeb60cc103a6cf13548155a228aedd52a35a9b063a641a85d4f

Right:
568f0248df3072297a3378af99c745fbca01267784a32b02eced2c2d8d0aaee1

Combined Hash ->
5bbb4c1536476567efb5c69c8a23a48ffb0dba92546465100b56250798f0a2c9

Pairing:

Left :
277bc788db649e93f8758a0fae636b92144adb5150a5fb82d98ac2c95ceae07f

Right:
fa69ffb869386437ea2645096d3e872652a01f9b65cfb52ff548a8ddf382ec50

Combined Hash ->
bfb366a79050805daf190bc52a57b2cefb3af18d2527680b4d05ace5b0e050f5

Pairing:

Left :
9e9a4c4243de110eb4b232d912351725ee6fcf1028f4f75638438ee83061feca

Right:
9e9a4c4243de110eb4b232d912351725ee6fcf1028f4f75638438ee83061feca

Combined Hash ->
296efe84d86b009bdbac09689c9294b7e40aeee5d08e79ed5ae17fe4eab42c74

[!] Odd count at Level 1. Duplicating last hash:
296efe84d86b009bdbac09689c9294b7e40aaaa5d08e79ed5ae17fe4eab42c74

=== Level 2: Intermediate Hashes ===

Pairing:

Left :
5bbb4c1536476567efb5c69c8a23a48ffb0dba92546465100b56250798f0a2c9
Right:
bfb366a79050805daf190bc52a57b2cefb3af18d2527680b4d05ace5b0e050f5
Combined Hash ->
a859a8844dec09ca57c47ec97b94213cb1159d530cba98e2887aac001e871287

Pairing:

Left :
296efe84d86b009bdbac09689c9294b7e40aaaa5d08e79ed5ae17fe4eab42c74
Right:
296efe84d86b009bdbac09689c9294b7e40aaaa5d08e79ed5ae17fe4eab42c74
Combined Hash ->
1acdf232dd1c130374eff3e9794f9467255af05a9f4669218d8c4873347be185

=== Level 3: Intermediate Hashes ===

Pairing:

Left :
a859a8844dec09ca57c47ec97b94213cb1159d530cba98e2887aac001e871287
Right:
1acdf232dd1c130374eff3e9794f9467255af05a9f4669218d8c4873347be185
Combined Hash ->
3027f97653180a7e0ed17e71fc56f2577b1e4b5c997bd8ffff3a3d21539da12d

=====

FINAL MERKLE ROOT HASH:

3027f97653180a7e0ed17e71fc56f2577b1e4b5c997bd8ffff3a3d21539da12d

=====

Conclusion -

In this experiment, four foundational blockchain scripts were debugged and executed. The Merkle Tree program was updated to process transaction data (T1T5) by hashing leaf nodes, pairing intermediate nodes recursively, and printing intermediate hash states. The lab demonstrated how Merkle Trees compress transaction data into a single Merkle Root Hash to enable efficient, tamper-evident verification.